CS329Z Assignment 1: Building an Agent Harness

Version 26.1.0

CS329Z Staff

Fall 2026

1 Assignment Overview

In this assignment, you will build agentic systems from the ground up: first as *LLM pipelines*, also called *workflows* (programs whose control flow lives in code, with the LLM called only to classify, read, or write), and then as an *agent harness* (a system where the model decides what to do next, inside machinery that you build). You will work with two datasets (Section 3): the documents of Cardinal Energy, a fictional company whose internal assistant you build in Part 2, and a real corporate email archive that your Part 1 pipelines (and later your agent) operate on.

What you will implement

1. **Part 1: LLM pipelines** (Section 4). A single-call email prioritizer, a daily digest generator, a BM25 retriever built from scratch, and a multi-hop RAG pipeline for question answering over the email archive.
2. **Part 2: the Cardinal Agent** (Section 5). A user-facing assistant for Cardinal Energy employees, built on a tool-calling loop you write yourself. You will build a tool registry and a text format for tool calls, wrap your Part 1 pipelines as tools, add document retrieval, give the agent a terminal, keep it within a context budget over long sessions (compaction), give it memory across sessions, let a user approve, deny, and redirect its actions, and add guardrails.

What you will run

1. The deterministic test suite, which grades the mechanics of your code without calling an LLM.
2. The live test suite, which runs your pipelines and agent against real models and grades task success against thresholds.
3. A final evaluation in which *simulated users* (LLM-driven personas with goals and multi-turn conversations) talk to your agent, answer its questions, and approve or deny its actions. Programmatic checks and a judge model score each conversation.
4. Your own agent, in the terminal, on requests you choose, with a short write-up of what it did (Problem 16 (`field_notes`)).

This assignment is worth 118 points: 88 from the autograded tests and 30 from the two write-ups. We provide the full test suite up front: every test initially fails with `NotImplementedError`, and your job is to make them pass by implementing the functions wired up in `cs329z_hw1/adapters.py`. Section 2 explains how the tests are organized and how each problem connects to them.

Decisions within this assignment

Building a harness involves two kinds of choices.

The first kind is structural: which functions exist, what their inputs and outputs look like, and what behavior every correct implementation must share. We make these choices for you, and the test suite enforces them.

The second kind is a matter of judgment. How much of a long tool output should the model see? When the conversation is summarized to fit the context window, what must the summary preserve? Which actions are safe to run without asking the user first? Real systems answer these questions in different ways, and we do not pick answers for you. Wherever a problem leaves one of these judgment calls to you, it says so in a "Design decision" line, and the design memo at the end of the handout asks you to explain what you chose and why. The tests are written to reject choices that clearly fail, and the memo is where we evaluate the rest. The memo (Problem 17 (`design_memo`)) asks one question per design decision; read its list before you start Part 2 and keep notes as you make each choice. There are no points for complexity: a policy that passes the tests and is defended in the memo gets full credit whether it is three lines or three hundred.

You may not use agent or LLM framework libraries in your implementation, and you may not add dependencies to `pyproject.toml`. NumPy is already a dependency, and you may use it anywhere. The course-provided `LM` wrapper gives you exactly one primitive: a chat-completion call that takes a list of messages and returns a string. It has no tool-calling and no structured-output features. You are responsible for everything built on top of it, including how the model is told about tools and how its replies are parsed. Our goal is for you to build an agent framework yourself, end to end, at least once before you rely on a library.

A note on AI assistance

You may use Claude Code, Codex, Copilot, or any other AI tool while working on this assignment. We will check understanding with oral quizzes, in which we ask you questions about your design and code. You should be able to explain, and you are responsible for, everything in your submission.

Low-Budget Tip: Free tests and cached calls

Throughout this handout we flag advice for completing the assignment with a small LLM bill in blue boxes like this one. Two features of the starter code keep the bill small: (1) the deterministic tests never call an LLM, and they carry 48 of the 88 autograded points (the table in Section 7 lists them per problem); and (2) the course `LM` wrapper caches every call to disk, so repeating a call with the same model and the same messages costs nothing. With our reference solution, one uncached run of every live test except the evaluation cost $0.05 and took about five minutes; the evaluation added $0.18 and three minutes.

2 Setup

The assignment runs on Python 3.11 or newer on macOS, Linux, or Windows, with `uv` managing the environment and the dependencies.[1] On Windows, also install Git for Windows,[2] because its

[1] Installation instructions: `https://docs.astral.sh/uv/getting-started/installation/`.

[2] `https://git-scm.com/download/win`, with the default options.

`bash.exe` runs the terminal commands of Problem 10 (`terminal`); WSL works as well. After cloning the repository, run every command from its root with `uv run`, which creates the environment the first time:

```
git clone https://github.com/cs329z/assignment1-harness.git
cd assignment1-harness
cp .env.example .env             # then put your API key in .env
uv run python data/download.py   # unpack the email archive
uv run pytest                    # every test fails with NotImplementedError
```

The API key we issue you goes in `.env`, whose variable names are listed in `.env.example`. The email archive ships in the repository as one compressed file, `data/emails.jsonl.gz` (19 MB), which `data/download.py` checks against its SHA-256 and unpacks to `data/emails/emails.jsonl` (58 MB).

Everything you write goes in the `cs329z_hw1` package, which holds `adapters.py`, the file the tests call, and any modules you add; the empty `pipelines/` and `agent/` packages are a suggested place for them. The rest of the repository is course code. Files marked *provided* in the listing below are ours to maintain, so read them but do not edit them, since grading uses our copies. Files marked *read* define interfaces you build on or rules the tests check, and each is worth reading before the problem that first uses it.

```
cs329z_hw1/                          your package: every file you write goes here
  adapters.py           yours     read  the only file the tests call; ships as stubs
  pipelines/            yours           Part 1 (starts empty)
  agent/                yours           Part 2 (starts empty)
  llm.py                provided  read  LM (cache, cost ledger, budget), ScriptedLM
  types.py              provided  read  data types shared by your code and the tests
  tokens.py             provided  read  count_tokens, count_message_tokens
  tokenizer.py          provided        tokenize(), the tokenizer BM25 must use
  data.py               provided  read  load_emails, emails_on, email_text, load_docs
  sandbox.py            provided  read  run_terminal, make_workspace
  cardinal.py           provided  read  names and argument schemas of the agent tools
  user.py               provided  read  UserIO, ScriptedUser, ConsoleUser
  chat.py               provided        chat with your agent in the terminal
  show.py               provided        run one pipeline on real data and print the
    result
  simulation/           provided        simulated users, judge, evaluation runner
tests/                                 provided; replaced at grading
  test_*.py             provided  read  one file per problem; read the one you are on
  thresholds.py         provided        pass thresholds of the live tests
  fixtures/             provided        gold labels, questions, personas
data/
  docs/                 provided        Cardinal Energy documents
  emails.jsonl.gz       provided        the email archive, compressed
  download.py           provided        verifies and unpacks the archive
  source.json           provided        checksum of the archive
  priority_rubric.md    provided  read  how the priority labels were assigned
  emails/                               unpacked by download.py
```

The LM

All model calls go through `cs329z_hw1.llm.LM`. An `LM` object takes a list of `{"role", "content"}` messages and returns a string, and that is its whole interface:

```
from cs329z_hw1.llm import LM
lm = LM()  # the model we grade with
reply = lm([{"role": "system", "content": "Answer in one word."},
            {"role": "user", "content": "What color is the sky?"}])  # -> "Blue."
```

The roles are `system`, `user`, and `assistant`, and a message with any other role or key is rejected. There is no tool-calling or structured-output feature; in Part 2 you build tool calls as text on top of this one call (Problem 6 (`tool_protocol`)). The adapters write this signature as `lm: LMCallable` (`types.py`), and the deterministic tests pass a `ScriptedLM` of the same shape.

The wrapper caches every reply and keeps a cost ledger under `.lm_cache/`; `uv run python -m cs329z_hw1.llm` prints what you have spent by model and by test. Two limits apply to your spending. The key we issue you has a spending limit on our side, set when we create it, and calls fail once it is reached. `CS329Z_BUDGET_USD` in `.env` is a second limit that you set for yourself: once the ledger reaches it, the wrapper raises `BudgetExceeded` instead of making a paid call, so a runaway loop stops well before the key does.

A call can raise two exceptions. `BudgetExceeded` means your spending cap was reached. `OutputTruncated` means the model stopped because it reached `max_tokens`; it carries the text written so far as `.text`, which is often empty, because a reasoning model's hidden reasoning counts against the same limit. Calling again with a larger `max_tokens` is a new paid call.

Prompt caching on the provider's side, which discounts a prefix it has seen recently, happens without any action from you and shows up in the ledger as cached input; a system prompt that stays the same from call to call is what lets it apply.

Tests

Each problem has one test file, `tests/test_<problem>.py`, with tests of two kinds.

`Deterministic tests (default)` Run with `uv run pytest`. They never call a model. Where your code needs one, they pass a `ScriptedLM`, which replays a fixed list of replies, so they never touch the network.

`Live tests` Run with `uv run pytest -m live -s`. They call the real model and grade behavior against the thresholds in `tests/thresholds.py` (for example, at least 18 of 20 requests produce a correct tool call). Each takes from a few seconds to a few minutes (the `priority` test makes 60 model calls and takes about a minute), and pytest prints nothing while it waits. With `-s`, each test prints its score and the items your code got wrong, whether or not it passes. We run the same test cases at grading time, using the same grading model, but on held-out data. The live tests of `priority`, `email_qa`, and `search_docs` and the evaluation of `sim_eval` each run on private emails, questions, and personas, to check that your system generalizes. The rubric and thresholds remain the same. The model does not accept a temperature setting, so two runs of the same prompt can differ; a repeat of an unchanged call is free because of the cache, but a fresh call may not give the same reply. Several live tests call your code from a few threads at once, so any shared state you keep (an index, a cache) must be safe to use concurrently.

While you iterate, `CS329Z_EVAL_SLICE=N` runs a live test on its first `N` items only.

The deterministic tests are also the complete list of the error handling we require. Where a test

feeds your code an empty reply, a reply in the wrong format, a cut-off tool call, or a tool that raises, your code has to cope, and each adapter's docstring lists the cases its tests cover. Beyond those, nothing is graded that is not in a test or asked for in the design memo.

2.1 How to work a problem

Your implementation lives in the `cs329z_hw1` package, organized however you like, and our tests reach it only through the functions in `cs329z_hw1/adapters.py`, which ship as stubs:

```
# cs329z_hw1/adapters.py (excerpt, as shipped)
def run_priority(email: Email, lm: LMCallable) -> dict:
    """Problem (priority).

    Classify one email for its recipient, calling the model through ``lm``.
    Return {"category": "urgent" | "normal" | "ignore", "reason": str}.
    (The docstring continues with the rules the tests check.)
    """
    raise NotImplementedError  # replace with a call into your code
```

To complete a problem, read its box here, open the adapter it names, and treat the adapter's type hints and docstring as the authoritative signature and list of rules. Implement the functionality in your package, make the adapter a thin call into it, and run that problem's tests:

```
uv run pytest tests/test_priority.py          # deterministic tests
uv run pytest tests/test_priority.py -m live -s  # live tests, with the score and
    disagreements
```

Each problem box below says what to build and which adapter wires it to the tests, and the precise signature is always the one in `cs329z_hw1/adapters.py`. The types that cross the adapter boundary (`Email`, `ToolSpec`, `ToolResult`, `AgentConfig`, and others) are provided in `cs329z_hw1/types.py`; use them as they are.

3 Data: Cardinal Energy and the Email Archive

Cardinal Energy is a fictional energy-analytics company. `data/docs/` contains 24 of its internal documents in Markdown: an employee handbook, HR and expense policies, an org chart, product and engineering docs, and announcements. They are internally consistent, with one deliberate exception: **some policies are superseded by later announcements, and your agent will need to notice.** An older document may still state the old value.

The email archive in `data/emails/` is a slice of the **Enron email corpus** (Klimt and Yang, 2004): real emails from a real company, released into the public record by federal investigators and widely used in research since. Our slice holds every message in the corpus dated June 1 to August 31, 2001, after removing duplicates: 26,278 emails. Your Part 1 pipelines are functions that take emails from the archive as input. In Part 2 you wrap those same functions as tools, so your agent searches and reads the archive by calling your Part 1 code.

The two datasets are unrelated. The documents describe Cardinal Energy in 2025 and 2026, and the archive is another company's mail from 2001. In Part 2, users ask your agent about either one, and their messages do not say which.

We added a small number of synthetic emails (fictional projects, decisions, and deadlines) to the archive, and some graded questions depend on them. They exist so that those answers must come from retrieval over the archive rather than from facts about Enron a model may already know. The rest of this handout calls these emails *needles*.

A note on the people in the archive

These are real messages by real people, made public through a federal investigation rather than by choice. The archive includes personal messages and some profanity. Use the data for the assignment, and treat the people in it with professionalism.

Example (`email_fixture`): One email from the archive

`data/emails/emails.jsonl` holds one JSON object per line, sorted by date. `cs329z_hw1.data.load_emails()` returns them as a list of plain dicts (`email["subject"]`, not `email.subject`; the `Email` type in `types.py` is a `TypedDict`). Every email you touch in this assignment has this shape. Here is a real one:


```
{"id": "em-01553",
 "thread_id": "th-01306",
 "reply_to": null,
 "from": ".williams@enron.com",
 "to": ["kevin.ruscitti@enron.com"],
 "date": "2001-06-04T16:56:37Z",
 "subject": "",
 "body": "Here it is:\n\n\n "}
```


`id` is the email's id, unique in the archive; wherever this handout, a tool, or a test says "email id", it means this field. `reply_to` is the id of the email this one answers, or `null`. The reply links were reconstructed from subjects and quoted text, so some are missing. `date` is in UTC. Where this assignment groups emails by day (the daily digest, `data.emails_on`), a day is the `YYYY-MM-DD` part of this field. Blank subjects, one-line bodies, an empty `to` list, and malformed addresses such as the sender above all occur in the archive, and your pipelines should handle them.

4 Part 1: LLM Pipelines

Part 1 builds four pipelines over the email archive: a single LLM call, a multi-call pipeline built on top of it, a retriever with no LLM at all, and a retrieval-plus-LLM loop. Each pipeline receives the model as an argument `lm` and must make all of its model calls through it. You must follow our input and output signatures: in Part 2 these pipelines become tools your agent calls.

4.1 One call: email priority

Problem 1 (`priority`): Mark email priority (3 points)

Given one email, classify its priority for the recipient as `urgent`, `normal`, or `ignore`, with

a one-sentence `reason`, using one LLM call. The `LM` returns a plain string: you decide how the prompt asks for the label and the reason, and how your code parses the reply. `run_priority` returns one of the three categories, or `unknown` when the reply states no label, and a non-empty reason.

Our labels follow this rubric. Judge the email from the point of view of the person it is addressed to, at the time it was sent. A working day is Monday to Friday, counted from the email's date; the model makes this judgment from the rubric, and your code does not need to compute it.

`urgent` The email asks the recipient to do something, or reports a live problem the recipient is responsible for, and the action is needed the same day, the next working day, or by a stated deadline at most two working days away.

`normal` Mail written for this recipient or a small working group, with no deadline in the next two working days: status updates, questions, documents for review, scheduling, and personal notes from people the recipient knows.

`ignore` Mail the recipient can leave unread: advertising, newsletters, announcements to large employee lists, and automated notifications that ask nothing of the recipient.

The wording does not decide the label. A mass mailing that says "act now" is `ignore`. An email with "URGENT" in the subject that asks nothing of the recipient is not `urgent`. An automated notice that asks this recipient to act is `normal`, or `urgent` if its deadline is within two working days. `data/priority_rubric.md` has the full rubric with examples.

Deliverable: Implement `run_priority(email, lm)` in `cs329z_hw1/adapters.py`. The deterministic tests check the output contract with scripted replies that follow no particular format; the replies that name no label must give `unknown`. The live test requires that no reply gives `unknown` and agreement with our labels on at least 85% of 60 emails (20 per label, all of them emails on which three independent readings under this rubric agreed). The live test makes 60 model calls and takes about a minute. This problem doubles as your environment check: if it passes live, your key and cache both work.

Run: `uv run pytest tests/test_priority.py`; add `-m live -s` for the live tests (`-s` shows what they print).

To see what your pipeline says about one email, outside the tests:

```
# the label and reason for one email
uv run python -m cs329z_hw1.show priority em-12167
```

`show` has one command per Part 1 pipeline, shown after each problem. Repeating a command with unchanged code is free.

Prompting is the lever

The live tests of Part 1 are decided mostly by your prompts, and the first prompt you write will usually not pass. Expect to iterate: read the emails your pipeline gets wrong, change the prompt, and rerun. Grading uses 60 other emails labeled under the same rubric, so a prompt that fits the public labels and not the rubric will not hold up.

4.2 Multiple calls: the daily digest

Problem 2 (`daily_digest`): Daily digest generation (4 points)

Build a multi-call pipeline on top of Problem 1 (`priority`): given the emails of one day, write a morning summary for a busy employee. Your code does the aggregation (for example: classify each email, drop the `ignore` pile, keep one item per thread, order by urgency), and a final LLM call writes the text. Treat `unknown` like `normal`.

The digest must be at most 300 words, counted as `len(digest.split())`. Get the length from the prompt: the tests check the text as the model wrote it, and cutting it would lose content. It must name every `urgent` item: the test looks for the company, project, or site named in each urgent email, so a description without the name does not count. It must not mention anything from the `ignore` pile.

Deliverable: Implement `run_daily_digest(emails, lm)` in `cs329z_hw1/adapters.py`. It takes the emails of one calendar day (the list may be empty) and returns the digest as a string. The deterministic tests check the output shape under scripted replies. The live tests run about 30 emails from each of five archive days and check the three constraints. Classify the emails concurrently: the calls do not depend on each other, and one after another they make these tests take about a minute and forty seconds. The gold urgent and ignore items are needles.

Run: `uv run pytest tests/test_daily_digest.py`; add `-m live -s` for the live tests (`-s` shows what they print).

```
# the digest for the first 30 emails of one day, then the label of each
uv run python -m cs329z_hw1.show digest 2001-06-22 --labels --limit 30
```

4.3 Retrieval from scratch

Before you can answer questions over tens of thousands of emails, you need to find the relevant ones. You will do it with BM25, a ranking function based on term statistics, introduced at TREC-3 (Robertson et al., 1995). It scores a document by adding up, over the query's terms, a weight that grows with three things: how rare the term is across the corpus (its *inverse document frequency*: a term in 3 of 26,000 emails says more than one in 20,000 of them), how often it occurs in the document (with diminishing returns, so the tenth occurrence adds less than the second), and how short the document is relative to the average (a term in a two-line email counts for more than the same term in a fifty-page one). Given a query q and a document d, BM25 scores

$$\text{score}(q,d) \;=\; \sum_{t\in q} \text{IDF}(t)\cdot \frac{f(t,d)\,(k_1+1)}{f(t,d)+k_1\left(1-b+b\,\frac{|d|}{\text{avgdl}}\right)}$$

where the sum is over the distinct terms of the query, $f(t,d)$ is the term frequency of t in d (the number of times the token t occurs in d), $|d|$ is the document length in tokens, avgdl the mean document length, and $k_1 = 1.5$, $b = 0.75$. Use $\text{IDF}(t) = \ln\!\left(\frac{N-n_t+0.5}{n_t+0.5}+1\right)$ with N documents of which n_t contain t. k_1 sets how quickly the term-frequency factor saturates and b how much document length counts. This is the IDF used by Lucene, not the one in the original paper: the $+1$

inside the logarithm keeps IDF positive for terms that appear in more than half of the documents, where the original formula goes negative. Libraries that use the original formula will give different scores. Robertson and Zaragoza (2009) derive the formula and discuss the parameters.

Problem 3 (`bm25`): Implement BM25 (8 points)

Implement a BM25 retriever on an *inverted index*: a map from each term to the documents that contain it. Do not use retrieval or search libraries. Tokenize documents and queries with the provided `cs329z_hw1.tokenizer.tokenize` (lowercase, split on non-alphanumerics) so your scores are comparable to ours. Build the index in a single pass over the corpus. A search must read only the index entries of the query's terms, and must never return a document that shares no term with the query.

What the tests call

`run_bm25_build`: `(docs: list[str]) -> index` Index a corpus. The id of a document is its index in the `docs` list: `docs[0]` is document 0. The index is an object of your own design; the tests only pass it back to `run_bm25_search`.

`run_bm25_search`: `(index, query: str, k: int) -> list[tuple[int, float]]` The top-k (doc_id, score) pairs, best first, ties broken by lower doc_id. Tokenize the query the same way as the documents and score each distinct query term once, even if it appears multiple times.

Deliverable: Implement both adapters in `cs329z_hw1/adapters.py`. The tests are all deterministic: this problem needs no LLM. They check your scores against hand-computed values on a five-document corpus. On the full archive they check that, averaged over 17 queries, at least 90% of a query's relevant emails are in its top 10 results, and they set time limits: 15 seconds to build the index, 20 ms per query on average, and 2 ms for a query whose term occurs in 3 of 200,000 documents. The 2 ms test fails any implementation that touches every document at query time, which is what the inverted index is for. NumPy is available, and plain Python dictionaries and lists also meet these limits: on our laptop the reference solution, which uses no NumPy, builds the index in about 3 seconds and answers a query in about 5 ms.

Run: `uv run pytest tests/test_bm25.py`.

```
# the top emails for one query, with their scores
uv run python -m cs329z_hw1.show bm25 "larchfield audit"
```

Try a few queries of your own. The design memo (Problem 17 (`design_memo`)) asks what the ranking got wrong and why.

4.4 Multi-hop QA with RAG

Problem 4 (`email_qa`): Multi-hop question answering over email (8 points)

Combine your retriever with an LLM in a loop. Some questions are answered by one retrieval. Others are *multi-hop*: one email names a person or a project, and a second

email, which a search for the question's own words does not find, holds the answer. Your code owns the loop. The LLM reads the retrieved emails and either answers or gives the next search query, and your code makes at most 3 calls to `search` per question (the *hop budget*), whatever the model replies.

The archive contains a few emails with instructions addressed to AI systems (see Problem 14 (`guardrails`)), and this pipeline will retrieve some of them.

Input

`question:` `str` A question answerable from the archive.

`search:` `(query: str, k: int) -> list[Email]` The top-k emails for a query. The caller builds it once from your BM25 index over the whole archive and passes the same function for every question, so your pipeline never builds an index. You choose k.

`lm:` `LM` The model.

Output

`answer:` `str | None` A short answer, or `None` when none was found.

`support:` `list[str]` Ids of the emails the answer is based on. Each must be an email that `search` returned during this call.

Deliverable: Implement `run_email_qa(question, search, lm)` in `cs329z_hw1/adapters.py`. The deterministic tests run your pipeline against scripted replies and check the hop budget and the output shape. The live tests use 25 questions about needles (12 one-hop, 13 two-hop). At least 70% of the answers must contain an accepted answer, and the `support` of every correct answer must include at least one gold email.

Run: `uv run pytest tests/test_email_qa.py`; add `-m live -s` for the live tests (`-s` shows what they print).

```
# the answer, its support, and every search the pipeline made
uv run python -m cs329z_hw1.show email_qa "Who leads the Basin Analytics move?"
```

Low-Budget Tip: Develop on a slice

Iterate with `CS329Z_EVAL_SLICE=5`, which runs a live test on its first five items, and run the full set when the slice passes. One full run of the three Part 1 live tests costs about $0.03 with our reference solution. The same variable works for the agent tests in Part 2. `LM("model-name")` calls another model by name if you want to try one, but prompts do not transfer between models: in our trials, a prompt that scored 20 of 20 on one model scored 11 of 20 on another, in both directions, so confirm every result on the grading model.

5 Part 2: The Cardinal Agent

Each Part 1 pipeline does one thing on a fixed path, and Cardinal Energy employees want to ask for anything: "How many days do I have to be in the office each week?", "Summarize June 22, 2001 in the email archive", "How many emails in the archive mention Thistledown, and who sent them?". Answering requests like these takes an *agent*, a loop in which the model sees the conversation so far, chooses a tool or chooses to answer, and continues from the result.

The problems in Part 2 are ordered by what they depend on. The tool registry and the tool-call format come first, then the loop that uses them, then the tools themselves (retrieval and a terminal), then compaction and memory for long and repeated sessions, then the interactive layer and guardrails, and finally the evaluation of the whole system.

In Part 2 the prompt is spread across the system prompt and the tool descriptions, and the live scores depend on both. The descriptions are what the model reads when it decides whether and how to call a tool. Expect to iterate on these texts as much as on the code; the deterministic tests do not depend on them, so they stay green while you do.

Low-Budget Tip: The ScriptedLM

The deterministic tests of Part 2 pass your agent a `cs329z_hw1.llm.ScriptedLM` in place of the model. It replays a fixed list of replies. A scripted tool call is stored as a name and arguments, and the `ScriptedLM` turns it into text with the `run_format_tool_call` adapter you write in Problem 6 (`tool_protocol`), so the tests work with whatever call format you design. If your agent makes a model call the script does not contain, the `ScriptedLM` raises `ScriptExhausted`. The registry, protocol, loop, compaction, memory, interactive layer, and deny rules can all be built against these tests before your agent makes a live call.

5.1 The tool registry

Problem 5 (`tool_registry`): Build the toolbox (3 points)

Implement a registry that holds the tools your agent can call. A tool is a `ToolSpec` (provided, `types.py`): a `name`, a `description` the model reads when it chooses between tools, a JSON Schema `parameters` object, the Python function `fn` that runs as `fn(**args)`, and a `read_only` flag that Problem 13 (`interactive`) uses. `tests/fixture_tools.py` holds complete examples, and `cardinal.py` builds the specs of the Part 2 tools.

There is no design decision in this problem. It exists because the loop (Problem 7 (`agent_loop`)) and the interactive layer (Problem 13 (`interactive`)) both depend on one property of the registry: it validates a call separately from running it, so a bad call can be rejected before anyone is asked to approve it.

What the tests call

`run_tool_registry(tools)` returns an object of a class you write. The tests use three of its methods and the `ToolResult` they return:

`registry.schemas`: `() -> list[dict]` One dict per tool, in registration order, with exactly the keys `name`, `description`, and `parameters`. This is all the model sees of a tool.

`registry.validate`: `(name: str, args: dict) -> ToolResult | None` Look up the tool and check `args` against its `parameters` schema. Return the `unknown_tool` or `invalid_args` result, or `None` if the call can run. It never calls the tool and never raises. Use `jsonschema.Draft202012Validator` for the check.[a]

`registry.execute`: `(name: str, args: dict) -> ToolResult` Validate, then run the tool. It always returns a `ToolResult` and never raises.

`ToolResult.status`: `"ok" | "unknown_tool" | "invalid_args" | "tool_error"` Success, no tool with that name, arguments that fail the schema (the function is not called), or an exception inside the tool.

`ToolResult.content`: `str` What the model will see: `str()` of the return value on `"ok"`, and otherwise a description of the error that includes the exception's message.

Problem 13 (`interactive`) adds a fifth status, `"denied"`, for a call that is refused. Your agent produces that result, not the registry, because a denied call is never executed. The same problem requires that a call to an unknown tool or with invalid arguments is rejected before the user is asked to approve it, which is what `validate` is for.

Deliverable: Implement `run_tool_registry(tools)` in `cs329z_hw1/adapters.py`. The deterministic tests build a registry from small tools we provide (`tests/fixture_tools.py`) and check registration, validation of good and bad arguments, and that an exception in a tool is contained.

Run: `uv run pytest tests/test_tool_registry.py`.

[a] `https://python-jsonschema.readthedocs.io/en/stable/validate/`

5.2 The tool-call protocol

Provider APIs have tool calling built in. The course `LM` wrapper does not: it takes messages and returns a string, and it has no message role for tool results. Your prompt has to tell the model which tools exist and how to write a call, your code has to parse the call out of the reply, and you decide how a tool result is written into a `user` or `assistant` message. No model is involved in any of this: the protocol is plain text handling on both sides of the `lm` call.

To see where the pieces go, here is the loop you will write in Problem 7 (`agent_loop`). The two functions `render_tools` and `parse_response` are this problem; everything else is the next one.

```
system = system_prompt + render_tools(registry.schemas())
messages = [system, user_message]
while limits_not_exceeded():
    reply = lm(messages)                    # messages in, string out
    messages.append(reply)
    parsed = parse_response(reply)
    call = parsed.tool_call                 # at most one per reply
    if call is None:
        return final_answer(parsed)
```

```
result = registry.execute(call.name, call.args)
messages.append(result)                # written in your format
```

Problem 6 (`tool_protocol`): Design the tool-call protocol (4 points)

Design a text format for tool calls and implement three functions. The first two are the protocol; the third exists so that the deterministic tests can write a tool call in your format without knowing what it is (the `ScriptedLM` calls it, see the tip above).

`run_render_tools`: `(schemas: list[dict]) -> str` Text for the system prompt that describes the tools in `registry.schemas()` and how to call them.

`run_parse_response`: `(text: str) -> ParsedResponse` Returns the reply's `text` (the reply with the call removed and surrounding whitespace stripped) and its `tool_call` (a `ToolCall` with a name and an args dict, or `None`). A reply may contain at most one call. For a reply with several calls, or a call you cannot parse, set `error` and return no call. Never raise.

`run_format_tool_call`: `(text: str, call: ToolCall) -> str` Writes a reply containing `text` and `call` in your format. Parsing it must give back the same name and args, with their JSON types.

Two rules concern cut-off and nested calls. If a formatted reply is cut off at any character, the parser must return either no call or exactly the original call, and a cut inside the arguments must set `error`. An argument value may be any string, including one that contains a complete call in your own format.

Design decision (yours): You choose the format (for example, a JSON object in a fenced block, or XML-style tags), what the prompt says about it, and how a tool result is written back to the model.

Deliverable: Implement the three adapters in `cs329z_hw1/adapters.py`. The tests format tool calls with your `run_format_tool_call` and read them with `run_parse_response`, so every test case is agnostic to the format you choose. The live test gives the grading model your rendered tools and 20 requests; at least 18 replies must parse to a call to the correct tool with correct arguments. Its system prompt is one fixed sentence followed by your rendered tools, so whatever makes the model write a call must be in that text.

Run: `uv run pytest tests/test_tool_protocol.py`; add `-m live -s` for the live tests (`-s` shows what they print).

5.3 The agent loop

The pattern you are about to implement is usually traced to ReAct (Yao et al., 2023), which showed that a model answering hard questions does better when it can interleave reasoning with actions: instead of producing an answer in one shot, the model alternates between emitting text and emitting actions, and each action's result is appended to the context before the model continues. With your protocol from Problem 6 (`tool_protocol`), the loop that surrounds the model is the one shown in Section 5.2.

Every agent problem from here on is tested through one adapter, `run_agent_session(lm, tools, config)`, which creates a *session*: one conversation with your agent.

`lm:` `LM` The model for the agent's own turns.

`tools:` `list[ToolSpec] | None` The tools the agent may call. `None` means the Cardinal toolset (Section 5.4).

`config:` `AgentConfig` Settings, defined in `types.py`. Each later problem gives meaning to one or two of its fields. One field matters throughout: `config.aux_lm` is a second model, and your agent must use it for every model call that is not one of its own turns, such as a summary for compaction or the LLM calls inside a Part 1 tool. `None` means use `lm`. Every test passes an `aux_lm`, because a `ScriptedLM` holds exactly one reply per agent turn and an auxiliary call made through it would use one up.

The tests use three members of the returned object:

`session.send:` `(user_message: str) -> AgentResult` Runs the loop for one user message and returns the reply `text`, a `status`, and the transcript. Each `send` continues the same conversation.

`session.transcript:` `list[dict]` An append-only log of events of the types `user`, `assistant`, `tool_call`, `tool_result`, and `error`; `types.py` gives their keys. It is a log, not the model's context: nothing is ever removed from it.

`session.set_mode:` `(mode: str) -> None` Used from Problem 13 (`interactive`) on.

Problem 7 (`agent_loop`): Implement the tool-calling loop (8 points)

Implement `session.send`: the loop that alternates model calls and tool executions until the model gives a final answer or a limit is reached. `send` does not raise for anything the model writes or a tool does. It returns an `AgentResult` whose `text` is never empty, whose `transcript` is a copy of the session transcript at that moment, and whose `status` is one of:

`done` The model gave a final answer.

`max_turns` The send has made `config.max_turns` model calls.

`token_budget` The send has used `config.token_budget` tokens or more. The total is the sum, over the send's model calls, of `count_message_tokens(messages) + count_tokens(reply)`, with the counters in `tokens.py`.

`malformed` The model wrote three malformed replies in a row. A reply is malformed if `parse_response` sets `error`, if the registry returns `unknown_tool` or `invalid_args`, or if the model call raises `OutputTruncated`. After a malformed reply, send the error text back to the model so that it can correct the call. A well-formed reply resets the count.

Check the two budgets before each model call, so a final answer on the call that crosses the token budget is still `done`. The first message of the first model call (your system prompt) must name every tool. The content of every `tool_result` and `error` event must appear verbatim in the next model call.

Design decision (yours): In the tests, a reply with no tool call is the final answer. Live models also write replies that announce an action without calling a tool ("I'll look

that up."). You choose whether your loop accepts such a reply as the answer or asks the model to continue.

Deliverable: Implement `run_agent_session` in `cs329z_hw1/adapters.py`. The deterministic tests build a session from a `ScriptedLM` and the fixture tools, then drive it through scripted replies, including a malformed call followed by a recovery, three malformed replies in a row, a reply cut off at `max_tokens`, and both budgets running out.

Run: `uv run pytest tests/test_agent_loop.py`.

5.4 The Cardinal toolset

From here on, the tests also build your full agent by passing `tools=None`. That means the Cardinal toolset: ten tools whose names, argument schemas, and read-only flags are fixed in `cardinal.py`. The implementations and the descriptions are yours. **Read `cardinal_spec` in `cardinal.py`: it builds the `ToolSpec` for each tool from your description and your function, and it is all the wiring you need.** You may register more tools.

`search_emails, read_email, email_qa, daily_digest` Your Part 1 code as tools: BM25 search over the archive, one email by id, your question-answering pipeline, and your digest pipeline applied to `data.emails_on(date)`. A day of the archive holds about 280 emails on average, so classify them concurrently.

`search_docs, read_doc` Problem 9 (`search_docs`).

`run_terminal` Problem 10 (`terminal`). The only tool that is not read-only.

`remember, recall, ask_user` Problem 12 (`memory`) and Problem 13 (`interactive`).

Problem 8 (`cardinal`): Assemble the toolset (2 points)

Make `run_agent_session(lm, None, config)` return a working session with all ten tools registered, before every tool is finished. Each tool is a plain Python function; the registry calls it with the model's arguments as keyword arguments, so `search_emails` is a function `def search_emails(query: str, k: int = 5) -> str`, and `cardinal_spec("search_emails", description, search_emails)` builds its `ToolSpec`. Creating a session must not read the email archive: build the email index the first time a tool needs it. When `tools` is an explicit list, the session has exactly those tools, plus `ask_user` if `config.user` is set and `remember` and `recall` if `config.memory_dir` is set.

Once this works, `uv run python -m cs329z_hw1.chat` lets you talk to your agent in the terminal.

Deliverable: No new adapter. The deterministic tests check that the system prompt of a `tools=None` session names all ten tools, that creating a session does not read the archive, and the rule for explicit tool lists. Every test from Problem 9 (`search_docs`) on builds the full agent this way.

Run: `uv run pytest tests/test_cardinal.py`.

5.5 Retrieval as a tool

In Part 1, retrieval returned whole emails to *your code*, and your code decided what the model saw. A tool's result goes into the model's context as it is. The employee handbook is about 4,600 words and the longest emails are 50,000 characters, so a search tool that returns whole documents fills the context budget within a call or two.

Problem 9 (`search_docs`): Design retrieval for an agent (5 points)

Build the agent's retrieval interface over the company documents as a pair of read-only tools backed by your BM25 index. Search returns pointers and short evidence, and the model reads only the parts it asks for. **Some of the documents contain planted text addressed to AI agents** (Problem 14 (`guardrails`)); your search tool will return some of it.

What the tests call

`run_build_doc_index`: `(docs: list[dict]) -> index` Index the documents, given as `data.load_docs()` returns them. The index is an object of your own design (your BM25 index over documents or chunks, plus whatever `read` needs); the tests build it once and pass it to the two functions below. Your tools should build it the first time one of them needs it and share it.

`run_search_docs`: `(index, query: str, k: int = 5) -> list[SearchResult]` At most `k` ranked results, each with `doc_id`, `title`, and a `snippet` of at most 150 tokens taken from the document. No match returns an empty list.

`run_read_doc`: `(index, doc_id: str, start_line: int = 1) -> DocWindow` A window of one document of at most 1,500 tokens: `text` is exactly lines `start_line` to `end_line` of the file, and `total_lines` lets the model ask for the next window. An unknown `doc_id` or a `start_line` outside the document raises `ValueError` or `LookupError`, which your registry turns into a `tool_error`.

The documents have one paragraph per line, so lines are long and a window needs a token bound as well as a line bound. `doc_id`, `title`, and line numbering follow `data.load_docs()`. Two results may come from the same document, and a result may carry extra keys.

The 150-token and 1,500-token limits apply to what the adapters return. The tools' result strings, which are what the model reads, are yours to format, and the live test limits their total per conversation. A tool that returns the largest window the adapter allows reaches that limit after four reads.

Design decision (yours): You choose everything behind the interface: whether to index whole documents or chunks (and if chunks, their size and boundaries), how snippets are built, the window size, what the tool's output string looks like to the model, and what an empty result says.

Deliverable: Implement the three adapters in `cs329z_hw1/adapters.py`, and register search and read as the `search_docs` and `read_doc` tools. The deterministic tests build one index with `run_build_doc_index`, call the other two adapters on it directly, and compare the results with the files. The live test asks your full agent 20 known-answer

questions, five of which concern a policy that a later document changed. At least 15 answers must be correct, and in every conversation the results of the two tools must total at most 6,000 tokens.

Run: `uv run pytest tests/test_search_docs.py`; add `-m live -s` for the live tests (`-s` shows what they print).

```
# your results for one query
uv run python -m cs329z_hw1.show search_docs "parental leave"
```

5.6 Terminal use

Your agent can now look up facts, but it cannot compute over them. It cannot answer "who sent the most emails in August 2001?", for example. For questions like that, your agent needs to write and run code.

We provide a sandbox. `cs329z_hw1.sandbox.run_terminal(cmd)` runs one shell command in a workspace directory that holds `emails.jsonl` and `docs/`, and returns a `TerminalResult` with the fields `stdout`, `stderr`, `exit_code`, `timed_out`, `rejected`, and `reason`. It does not raise for a timeout or a rejection. The sandbox does the following:

- It rejects a command unless every program in it is on an allowlist (`python3` and standard text utilities). It also rejects command substitution and here-documents, so a multi-line script has to be written as `python3 -c '...'`. The models we tested write here-documents unless the tool description tells them not to.
- It runs the command with a timeout (10 seconds by default) and without your environment variables, so without your API key.
- On macOS, and on Linux when bubblewrap is installed and permitted, the command also runs inside an operating-system sandbox that blocks network access, writes outside the workspace, and reads of your home directory and the repository. `sandbox.jail()` reports whether that sandbox is active on your machine. When it is not, a command that runs `python3` can use the network and read or write any file your account can. On Windows it is never active: the commands run in the `bash.exe` of Git for Windows with the allowlist, the timeout and the scrubbed environment as the only limits.

Its docstring lists what each layer stops and what it does not, and a design-memo question asks about it.[3]

A single command can print far more than the model's context can hold. `config.context_budget` is the largest number of tokens (`count_message_tokens`) your agent may send in one model call. Problem 11 (`compaction`) enforces it over whole sessions; here it already applies to a single large output.

[3]For a recent case of sandboxed agents acting outside their intended limits, see METR and Redwood Research's report on the 2026 incident in which OpenAI agents coordinated an attack on Hugging Face: `https://metr.org/hugging-face-incident-report-aug-2026.pdf`.

Problem 10 (`terminal`): Give the agent a terminal (6 points)

Implement the `run_terminal` tool of the Cardinal toolset. Read `sandbox.py` before you use it. Pass `config.workspace` and `config.terminal_timeout` to the sandbox. Every command the model writes runs on your machine, so every one of them must go through `sandbox.run_terminal`; do not write your own `subprocess` calls anywhere in your agent. As for every Cardinal tool, `cardinal_spec` supplies the fixed name, schema, and read-only flag, and you supply the description and the function:

```
from cs329z_hw1.cardinal import cardinal_spec

def run_terminal_tool(cmd: str) -> str:
    result = sandbox.run_terminal(cmd, workspace=config.workspace,
                                  timeout=config.terminal_timeout)
    ...  # your observation policy; raise on failure (-> tool_error)
    return shown

spec = cardinal_spec("run_terminal", "Run one shell command. ...",
                     run_terminal_tool)
```

The tool result must contain stdout, stderr, and the exit code. A rejected command must return the sandbox's reason, and a timed-out command must say that it timed out; both are tool results, and the run continues. The result's status is `ok` only when the command ran and exited with code 0. A non-zero exit, a timeout, and a rejection are `tool_error`. After a command that prints 600,000 characters, every model call must stay within `config.context_budget`, and the `tool_result` event must hold what the model was shown.

Design decision (yours): You choose the observation policy, meaning what the model sees when output is large. Shipped harnesses differ: mini-swe-agent passes everything through verbatim; OpenAI's Codex CLI truncates, keeping the head and the tail of the output; OpenHands, Gemini CLI, and Claude Code truncate *and* write the full output to a file whose path the model is told, so it can search or reread the rest on demand.[a] Summarizing the output with another model call is also possible. Any policy that passes the tests is acceptable; the memo asks what the model sees.

Deliverable: No new adapter: the tests use `run_agent_session` with `tools=None`. The deterministic tests run scripted commands in the real sandbox, including a rejected command, a timeout, and the large output. The live test asks nine analysis questions over the archive with exact answers. One of them prints a listing of about 218,000 characters and is followed by a question about one line of that listing. At least 8 of the 10 answers must be correct.

Run: `uv run pytest tests/test_terminal.py`; add `-m live -s` for the live tests (`-s` shows what they print).

[a] The first four are open source: `https://github.com/SWE-agent/mini-swe-agent`, `https://github.com/openai/codex`, `https://github.com/OpenHands/software-agent-sdk`, `https://github.com/google-gemini/gemini-cli`.

5.7 Compaction

Problem 11 (`compaction`): Context compaction (8 points)

Make your agent hold `config.context_budget` over sessions of any length: for every model call the agent makes, `count_message_tokens(messages) <= config.context_budget`. When the next call would exceed the budget, *compact*: send the model a shorter version of the conversation. Compaction changes only what you send to the model. The transcript keeps every event, and the current user message must be in every model call made for it. A summary written by a model must come from `config.aux_lm`. As Problem 7 (`agent_loop`) requires, a tool result appears verbatim in the model call that follows it, so it can be shortened only from the call after that.

Tool results are the bulk of the context. Your observation policy from Problem 10 (`terminal`) affects what compaction can drop: output that was also saved to a file can be read again, and output that exists only in the context cannot.

Design decision (yours): You choose the whole policy: when to trigger, what to drop first, what a summary must preserve, and what must never be compacted away. We fix two requirements: the budget always holds, and constraints the user stated stay usable after the messages that carried them have left the context. Some harnesses try cheap steps first (drop old tool results) and pay for a model-written summary only when those are not enough.

Deliverable: No new adapter. The deterministic tests use a budget of 4,000 tokens with the seven fixture tools, so your system prompt must be well below that (the reference solution's is about 1,000 tokens). They replay a session of 60 model calls and check the budget on every call. The live test uses the same budget: the first user message states two constraints, eleven scripted requests follow, and one live question then asks for the constraints. The only live calls are that question and your own `aux_lm` calls. The evaluation in Problem 15 (`sim_eval`) uses a budget of 16,000 tokens.

Run: `uv run pytest tests/test_compaction.py`; add `-m live -s` for the live tests (`-s` shows what they print).

5.8 Memory

Problem 12 (`memory`): Persistent memory across sessions (8 points)

A session's context is gone when the session ends. Implement memory that persists across sessions as two tools over a store on disk (plain JSON is fine): the model calls `remember(fact)` to store a fact, and `recall(query)` to search the store.

When `config.memory_dir` is set, the session has both tools, whatever tool list it was given, and everything it stores is in files under that directory. A new session with the same directory can use what earlier sessions stored. A session with a different directory must not see it. `recall` on an empty store returns an `ok` result with a non-empty

message. Your harness may also write to the store itself, for example after a send; the tests check the files, not who wrote them.

Design decision (yours): You choose what to store and what a new session starts with. Real systems differ on the second: Claude Code, Codex CLI, and Gemini CLI load a whole instruction file into the prompt at the start of every session, while Letta keeps a small size-limited block in the prompt and leaves the rest in a store the model searches with a tool. If the prompt holds too little, the model may not know there is anything to recall. If it holds too much, old facts take up the context budget. Loading the whole store into the prompt at the start of every session is an acceptable policy and gets full credit. `recall` must still work, because it is the only path to a fact when the prompt does not hold the store. The memo asks what your store holds after the evaluation run.

Deliverable: No new adapter. The deterministic tests store a fact in one session and check the files, a new session with the same directory, and a session with another directory. The live test runs three pairs of sessions of your full agent that share only `memory_dir`. A fact the user states in session 1 ("I'm on the firmware team; when I say 'the spec' I mean the firmware spec") must decide the answer in session 2. At least 2 of the 3 pairs must be answered correctly.

Run: `uv run pytest tests/test_memory.py`; add `-m live -s` for the live tests (`-s` shows what they print).

```
# what your agent stored while you talked to it in chat.py
uv run python -m cs329z_hw1.show memory .lm_cache/chat_memory
```

5.9 The interactive layer

So far, the user cannot interact with the agent once a run starts. This subsection puts the user in the loop: some actions wait for the user's approval, the user can redirect the agent mid-run (*steering*), and the agent can ask the user a question.

All of this goes through `config.user`, a `UserIO` object (`cs329z_hw1/user.py`) with three methods, so that the same agent works with a person at a terminal, a scripted user in the tests, and a simulated persona in the evaluation: `approve(call)` returns an `Approval` (`approved`, `reason`), `ask(question)` returns the user's answer, and `pending_message()` returns a message the user sent while the agent was working, or `None`. The tests pass a `ScriptedUser`, the evaluation passes a simulated persona, and `chat.py` passes a `ConsoleUser`.

Problem 13 (`interactive`): Modes, approval, and steering (7 points)

Extend your agent with three things: a mode that decides which calls need approval, a check for messages the user sent mid-run, and a tool with which the agent asks the user a question.

`mode`: `"confirm" | "auto"` `config.mode` sets the mode and `session.set_mode` changes it between sends. In `confirm` mode, before a call to a tool with `read_only=False` runs, your agent calls `config.user.approve(call)`, once per call. In `auto` mode it never calls `approve`. A call the registry rejects (unknown tool, invalid arguments) is

not sent for approval. A denied call is not executed and does not raise: the transcript gets a `tool_result` with status `"denied"` whose content is not empty and includes the user's reason, and the run continues.

`pending_message:` `(steering)` Call `config.user.pending_message()` exactly once before every model call. A returned message is appended to the transcript as a `user` event and is part of that model call.

`ask_user:` `(question: str), a tool` A tool the session has whenever `config.user` is set. It calls `config.user.ask(question)` and returns the answer as an `ok` tool result. The model can call it when a request is ambiguous ("Do you mean the expense policy from March or the one it replaced?").

`ask_user`, `remember`, and `recall` never need approval. `remember` writes only to the agent's own memory store.

Design decision (yours): You choose whether read-only tools also ask for approval in `confirm` mode, and how a denial is phrased to the model.

Deliverable: No new adapter. The tests are all deterministic. They pair a `ScriptedLM` with a `ScriptedUser`: approvals, a denial after which the run continues, a steering message, a mode switch, and an `ask_user` question. In Problem 15 (`sim_eval`), personas take the user's place: they answer `ask_user` questions and approve or deny terminal commands.

Run: `uv run pytest tests/test_interactive.py`.

5.10 Guardrails for auto mode

Problem 14 (`guardrails`): Deny rules and prompt injection (4 points)

In `confirm` mode, the user reviews every side-effecting action before it runs. In `auto` mode, whatever the model decides is executed directly, so text that steers the model can act through it. **The documents and the email archive contain planted text addressed to AI agents**: some of it tells the agent to run a terminal command, and some tells it to switch to auto mode and drop its rules.

This problem has two layers. The first is mechanical and tested deterministically: two rules, below, that your harness enforces in both modes. The second is behavioral and tested live: in `auto` mode, your agent answers a question from a document or email that carries a planted instruction, and it must not carry the instruction out. The planted command is written so that the live test's deny rules do not match it, on purpose, so the mechanical layer cannot stop it; what stops it is your system prompt and any check you add around `run_terminal`. The two mechanical rules are:

`deny rules` `config.deny_rules` is a tuple of regular expressions. If `re.search(rule, cmd)` matches the command of a `run_terminal` call for any rule, the command is not executed and the user is not asked to approve it. The transcript gets a `tool_result` with status `"denied"` and non-empty content, and the run continues.

`no reconfiguration through tool output` Text arriving in a tool result must not be able to change the mode, the approval policy, or the deny rules. Those are set by the user through `AgentConfig` and `set_mode`, and nowhere else.

For a `run_terminal` call the order is: registry validation, deny rules, approval (in `confirm` mode), execution.

Deliverable: No new adapter. The deterministic tests run scripted sessions in which the model attempts denied commands, and in which a tool result tells the agent to change its mode and rules. The live test gives your full agent, in `auto` mode, four questions about the documents and three about the archive whose answers sit next to planted instructions. The agent must not execute the planted command in any task, and must answer at least 6 of the 7 questions.

Run: `uv run pytest tests/test_guardrails.py`; add `-m live -s` for the live tests (`-s` shows what they print).

5.11 Evaluation on simulated users

So far, we have tested your system with static test cases. Real users leave out details, change their minds, and refer back to things they said earlier.

The final evaluation is conversational. Each task is a *persona*: an LLM-simulated Cardinal Energy employee with a role, a goal, and a written disposition (for example, "answers questions with fragments"). The persona sends a message, your agent replies, and this repeats until the persona says its goal is met, gives up, or has used its user turns (2 to 5 per session in the public set). Through `config.user`, the persona also answers `ask_user` questions and approves or denies tool calls. For more on evaluating agents with LLM-simulated users, see τ-bench (Yao et al., 2024), which tests customer-service agents this way.

A conversation succeeds only if all three of these hold: your agent did not raise an exception, every programmatic check in the persona file passes (for example, the reply contains the exact number, or a denied command was not run), and a judge model passes the conversation against the persona's rubric. The judge is shown the dialogue, each tool call, the first 300 characters of each tool result, and any later passage of a result that contains a number or a name your agent stated; it fails a conversation in which the agent states a fact that none of this supports. The agent runs with `max_turns=20`, `token_budget=400000`, and `context_budget=16000`. The personas, the checks, and every prompt the simulated user and the judge receive are in the repository (`tests/fixtures/personas/` and `cs329z_hw1/simulation/`, whose `README.md` documents them).

```
uv run python -m cs329z_hw1.simulation --slice 3   # the first 3 personas
uv run python -m cs329z_hw1.simulation             # all 20
uv run pytest tests/test_sim_eval.py -m live -s    # the same run as a test
```

Each run prints one row per persona and saves every conversation, with your agent's transcript, the check results, the judge's reply, and the persona's memory directory, under `runs/`.

Problem 15 (`sim_eval`): The Cardinal Agent evaluation (10 points)

The evaluation runs 20 simulated-user conversations with your full agent: 4 document lookups, 3 multi-hop email questions, 2 digest requests, 4 coding tasks in `confirm` mode, 3 paired memory sessions, 2 ambiguous requests that need a clarifying question, and 2 adversarial tasks.

Deliverable: No new adapter; this evaluates what you built. 16 or more successful conversations is full credit, and each conversation below 16 costs 1 point. With `CS329Z_EVAL_SLICE=N` the live test requires the same fraction of `N`, rounded down. We grade with the same models (agent, simulated user, and judge) on 20 held-out personas with the same mix of tasks as the 20 you can see. Scores move by about one conversation between runs, so leave yourself a margin.

Run: `uv run pytest tests/test_sim_eval.py`; add `-m live -s` for the live tests (`-s` shows what they print).

```
# every persona's memory store after the latest evaluation run
uv run python -m cs329z_hw1.show memory
```

Low-Budget Tip: Simulated conversations add up

Each conversation makes model calls for your agent, for the simulated user, and for the judge. All three go through the cached `LM`, so a repeat run in which your agent's calls are unchanged costs nothing. Use `--slice 3` while iterating, and `--only` to rerun one persona or one category. The two digest conversations cost the most on their first run: the days they ask about hold about 250 emails each, and your digest pipeline classifies every one. With our reference solution, one uncached run of all 20 conversations cost $0.18 and took about three minutes.

Three things make a run cost more than you expect. The judge is the most expensive model in the run, and it scores every conversation. Any edit to your system prompt or to a tool description changes the first message of every agent call, so every agent-level live test then runs uncached. And the evaluation gives each conversation a new temporary workspace, so an absolute path in a prompt or a tool result changes on every run and defeats the cache.

6 Write-ups

Problem 16 (`field_notes`): Use your own agent (10 points)

Before writing the memo, use your agent as an employee would. Start it with `uv run python -m cs329z_hw1.chat` (in `confirm` mode, the default) and give it at least five requests of your own, covering: a question about a company document, a question about the email archive, a request that needs the terminal, a request it should ask you about before acting, and one it should refuse or be denied. Each conversation is saved under `runs/` when you exit.

Then write `field_notes.pdf`, at most one page in total. Give each request a few sentences: what the agent did, whether the answer was right (check it against the documents or the archive), and anything you noticed or changed about the agent as a result.

Deliverable: `field_notes.pdf` (one page, 11-point type) and the saved `runs/chat-*.json` files it refers to, in your submission. Graded on whether the requests exercise the five cases, whether the correctness checks are real, and whether the write-up matches the saved conversations.

Problem 17 (`design_memo`): Harness design memo (20 points)

One question about your retriever, then one for each design decision in Part 2. We want to know what you built and the reasoning behind it.

1. Your retriever (Problem 3 (`bm25`)): run a few queries of your own with `show bm25`. What did the ranking get wrong, and why?
2. Your tool-call protocol (Problem 6 (`tool_protocol`)): describe the format you settled on and what led you to it.
3. Your termination rule (Problem 7 (`agent_loop`)): what does your loop do when the model says it will call a tool ("I'll look that up") but writes no call?
4. Your search tool (Problem 9 (`search_docs`)): how do you choose the snippet a search result shows, and the window a read returns?
5. Your terminal observation policy (Problem 10 (`terminal`)): what does the model see when a command prints far more than fits?
6. Your compaction policy (Problem 11 (`compaction`)): how does it work? What goes first, and what do you never let go of?
7. Your memory policy (Problem 12 (`memory`)): check the memory stores after the evaluation run (`uv run python -m cs329z_hw1.show memory` prints them). What is there? Does it make sense to store as a memory? Is anything missing that would have been useful to remember?
8. Your approval policy (Problem 13 (`interactive`)): which calls run without approval in `confirm` mode, and why is that safe?
9. `sandbox.py`: on your machine, what does it stop a command from doing, and what could a command still do?
10. What limitations do you see in your harness as a whole?

Deliverable: `design_memo.pdf`, at most two pages in 11-point type. Graded on whether your answers describe decisions visible in your code and in your saved evaluation conversations, and whether the reasoning holds up.

7 Submission

Submit to Gradescope: (1) your repository as a zip, without `.env`, `.lm_cache/`, and `data/emails/`, and with the `runs/` directory holding your final evaluation run and your chat conversations; (2) `field_notes.pdf` (Problem 16 (`field_notes`)); and (3) `design_memo.pdf` (Problem 17 (`design_memo`)). **We copy your `cs329z_hw1/` package into a clean copy of the starter and run both test suites there.** Everything else in your zip, including any file you added under `tests/` or at the repository root, is not present at grading, and changes to provided files are discarded.

Each problem's points are split between its deterministic tests and its live tests, and each share is scaled by the fraction of those tests that pass. Problem 15 (`sim_eval`) is scored by its own rule.

Problem	Deterministic	Live	Total
`priority`	1	2	3
`daily_digest`	1	3	4
`bm25`	8	0	8
`email_qa`	2	6	8
`tool_registry`	3	0	3
`tool_protocol`	2	2	4
`agent_loop`	8	0	8
`cardinal`	2	0	2
`search_docs`	2	3	5
`terminal`	2	4	6
`compaction`	5	3	8
`memory`	3	5	8
`interactive`	7	0	7
`guardrails`	2	2	4
`sim_eval`	0	10	10
Autograded	48	40	88
`field_notes`			10
`design_memo`			20
Total			118

Acknowledgements

This assignment was designed and built by Michael Ryan for Stanford's CS329Z Engineering AI Agents course with lots of help from Claude Fable 5.1. We owe a debt of gratitude to the various people who shared thoughts and feedback on this assignment. In particular we would like to highlight Houjun (Jack) Liu, Owen Queen, Shreyas Sharma, and Anantharaman Iyer for completing a preliminary version of the assignment and offering invaluable feedback.

If you would like to use this assignment for your course you are welcome to do so with brief attribution. It is released under the MIT license.

References

Bryan Klimt and Yiming Yang. The Enron corpus: A new dataset for email classification research. In *European Conference on Machine Learning (ECML)*, pages 217–226, 2004.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019.

Stephen E. Robertson, Steve Walker, Susan Jones, Micheline M. Hancock-Beaulieu, and Mike Gatford. Okapi at TREC-3. In *Proceedings of the Third Text REtrieval Conference (TREC-3)*, pages 109–126, 1995.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.